

Smart Supermarket Billing System

A Python-based billing and analytics solution designed to automate retail transactions, apply dynamic discounts, and generate actionable business insights from sales data.

Presented By :
Prakash Chawda



Scan To Learn More



Supermarket Billing Software



Problem

Traditional supermarket billing systems lack analytical capabilities and efficient automation. Manual or basic systems fail to handle multiple items efficiently, apply dynamic discount strategies, or provide real-time insights into sales performance.



Manual Billing

Basic systems fail to handle multiple items efficiently



No Dynamic Discounts

Cannot apply dynamic discount strategies



No Real-Time Insights

Fail to provide real-time insights into sales performance



No Structured Reports

Fail to generate structured reports for decision-making

Objective

Automate Billing

Automate the billing process for multiple products

Dynamic Discounts

Implement dynamic discount rules

Itemized Bills

Generate itemized bills with accuracy

Method



Input Collection

Take item name, quantity, and price from user



Processing

Calculate total price and apply discount using condi



Billing Generation

Display itemized bill with all details



Output

Generate final bill and summary report





Supermarket Billing System :

```
# ## Implementation
from datetime import datetime
def supermarket_billing_system():
    """
    Main function to run the supermarket billing system.
    Handles item input, discount calculation, and receipt generation.
    """

    # Initialize lists to store billing data
    items = []          # Store item names
    quantities = []     # Store quantities
    prices = []         # Store unit prices
    item_totals = []    # Store total for each item (quantity x price)
    date_time = datetime.now().strftime("%d/%m/%Y %H:%M")

    print("\n" + "="*50)
    print("      WELCOME TO SUPERMARKET BILLING SYSTEM")
    print("="*50)
```


Input Value :



```
# Loop to accept multiple items (Key Requirement: Use loops for continuous input)
while True:
    print("\n--- Enter Item Details ---")

    # Get item name
    item_name = input("Item name (or 'done' to finish): ").strip()

    # Check if user wants to finish billing
    if item_name.lower() == 'done':
        if len(items) == 0:
            print("\nNo items entered. Exiting system.")
            return
        break

    # Get quantity with validation
    while True:
        try:
            quantity = int(input("Quantity: "))
            if quantity <= 0:
                print("Quantity must be positive. Please try again.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a valid number.")
```



Input Values & Append :

```
# Get unit price with validation
while True:
    try:
        price = float(input("Unit price (₹): "))
        if price < 0:
            print("Price cannot be negative. Please try again.")
            continue
        break
    except ValueError:
        print("Invalid input. Please enter a valid price.")

# Calculate total for this item
total = quantity * price

# Store data
items.append(item_name)
quantities.append(quantity)
prices.append(price)
item_totals.append(total)

print(f"✓ Added: {quantity} x {item_name} @ ₹{price:.2f} = ₹{total:.2f}")
```

Discount Offer :



```
# Calculate subtotal
subtotal = sum(item_totals)

# Apply discounts based on total amount (Key Requirement: Conditional statements)
discount_percentage = 0
discount_amount = 0

if subtotal >= 5000:
    discount_percentage = 15
    discount_amount = subtotal * 0.15
    discount_reason = "15% discount (Purchase above ₹5000)"
elif subtotal >= 3000:
    discount_percentage = 10
    discount_amount = subtotal * 0.10
    discount_reason = "10% discount (Purchase above ₹3000)"
elif subtotal >= 1500:
    discount_percentage = 5
    discount_amount = subtotal * 0.05
    discount_reason = "5% discount (Purchase above ₹1500)"
elif subtotal >= 500:
    discount_percentage = 2
    discount_amount = subtotal * 0.02
    discount_reason = "2% discount (Purchase above ₹500)"
else:
    discount_reason = "No discount applicable"

# Calculate final total
final_total = subtotal - discount_amount
```




Printing Bill :

```
# Generate and display itemized receipt (Key Requirement: Itemized bill)
print("\n" + "="*60)
print("                BILL RECEIPT")
print("="*60)
print(f"{'Item':<20} {'Qty':<6} {'Price(₹)':<10} {'Total(₹)':<12}")
print("-"*60)

for i in range(len(items)):
    print(f"{items[i]:<20} {quantities[i]:<6} ₹{prices[i]:<9.2f} ₹{item_totals[i]:<11.2f}")

print("-"*60)
print(f"{'Subtotal':<36} ₹{subtotal:<11.2f}")
print(f"{'discount_reason':<36} -₹{discount_amount:<10.2f}")
print("-"*60)
print(f"{'FINAL AMOUNT DUE':<36} ₹{final_total:<11.2f}")
print("="*60)
```



Payment Processing :

```
# Payment processing
print("\n--- Payment ---")
while True:
    try:
        amount_paid = float(input("Amount paid (₹): "))
        if amount_paid < final_total:
            print(f"Insufficient amount. Need ₹{final_total - amount_paid:.2f} more.")
            continue
        break
    except ValueError:
        print("Invalid input. Please enter a valid amount.")

change = amount_paid - final_total
print(f"\nChange to return: ₹{change:.2f}")
print("\n" + "="*60)
print("    THANK YOU FOR SHOPPING WITH US!")
print("    Please visit again!")
# ("PRAKASH CHAWDA")
print("Date" , date_time)
print("="*60 + "\n")

# Run the billing system
if __name__ == "__main__":
    supermarket_billing_system()
```



Sample Input :

```
=====
WELCOME TO SUPERMARKET BILLING SYSTEM
=====

--- Enter Item Details ---
Item name (or 'done' to finish): Apple
Quantity: 25
Unit price (₹): 100
✓ Added: 25 x Apple @ ₹100.00 = ₹2500.00

--- Enter Item Details ---
Item name (or 'done' to finish): Orange
Quantity: 20
Unit price (₹): 60
✓ Added: 20 x Orange @ ₹60.00 = ₹1200.00

--- Enter Item Details ---
Item name (or 'done' to finish): Banana
Quantity: 20
Unit price (₹): 40
✓ Added: 20 x Banana @ ₹40.00 = ₹800.00

--- Enter Item Details ---
Item name (or 'done' to finish): done
```

Sample Output :

```
=====
BILL RECEIPT
=====
Item                Qty    Price(₹)    Total(₹)
-----
Apple                25      ₹100.00    ₹2500.00
Orange               20      ₹60.00     ₹1200.00
Banana               20      ₹40.00     ₹800.00
-----
Subtotal                                ₹4500.00
10% discount (Purchase above ₹3000)  -₹450.00
-----
FINAL AMOUNT DUE                                ₹4050.00
=====

--- Payment ---
Amount paid (₹): 4500

Change to return: ₹450.00

=====
THANK YOU FOR SHOPPING WITH US!
Please visit again!
Date 14/04/2026 09:40
=====
```

What We Accomplished



Fully Functional System

Built a fully functional billing system



Real-Time Logic

Integrated real-time calculations and discount logic



Structured Data

Generated structured datasets for analysis



Actionable Insights

Provided actionable insights for business decisions

Future Opportunities



SQL Integration

Integration with SQL database for storage



Dashboard Development

Dashboard development using Power BI or Tableau



AI Forecasting

AI-based demand forecasting



Barcode System

Barcode scanning system



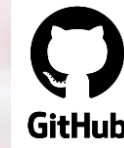
Inventory Tracking

Real-time inventory tracking

WsCube-Tech



[LinkedIn.com](https://www.linkedin.com)



[GitHub.com](https://github.com)

Conclusion

The Smart Supermarket Billing System successfully demonstrates how Python can be used to automate retail billing while generating valuable business insights. This project bridges the gap between operational efficiency and data-driven decision-making, making it highly relevant for real-world retail environments.